Reverse-engineering a 1997 modem with an LLM as a junior collaborator

What worked, what didn't, and the bug that needed a human

---

honx

38C3 — Hamburg

Disclaimers

What this talk is *not*

- "AI replaces reverse engineers."
- "You can prompt your way to a working tool."
- "LLMs are about to make Ghidra obsolete."
- Vendor pitch. There is no product here. Just a git repo.

What this talk is *not*

- "AI replaces reverse engineers."
- "You can prompt your way to a working tool."
- "LLMs are about to make Ghidra obsolete."
- Vendor pitch. There is no product here. Just a git repo.

What it is: four days of pair-programming with an LLM on a specific RE task, with the receipts. Including the part where I got something wrong and someone with grey hair fixed it.

The deal

- Audience: you, sceptical.
- Subject: AI-assisted RE, on a real chip with real datasheets.
- Outcome metric: a working Ghidra module, a tested SLEIGH spec, and a 350-line analysis writeup. Or not. We'll see.
- Counterweight: the bug that proves the AI didn't really understand what it was doing.

The artefact

A piece of 1997 hardware

ELSA MicroLink 33.6 TQV

- German consumer modem, 1997
- 33.6 kbps V.34 + V.42bis + LAPM
- Class 2 fax (T.30)
- +V voice mode (V.253)
- H.324 video over POTS

Inside

- Rockwell L3902 MCU
- RC240VFC analog front-end
- 128 KiB external EPROM
- ∼32 KiB external SRAM

Why this thing? I had the firmware blob. The chip's CPU is 6502-derived but *not* 6502, and Ghidra ships nothing for it.

The CPU: Rockwell R65C19

Enhanced 6502, late 1980s.

On top of stock 65C02:

- 12 new bit-manip / branch ops
- 21 new arithmetic ops (`MPY`, `MPA`, `NEG`, `ASR`, `ADD`, ...)
- 10 threaded-code ops (`NXT`, `TIP`, `JPI`, ...)
- 16-bit `W` reg (multiply accumulator)
- 16-bit `I` reg (Forth-style instruction pointer)

The trap nobody mentions

Two addressing modes are silently redefined:

- `(zp,X)` → `(zp)`
- `(zp),Y` → `(zp),X`

Same opcodes. Different semantics.

Loading as 65C02 fails about 1 KiB in.

Banking

The L3902 talks to its 128 KiB EPROM through 8 bank-select registers at `$0018–$001F`. Each one maps a single 8 KiB physical bank into one of eight 8 KiB windows in the CPU's 64 KiB view.

```
BSRn = | ES3 | ES2 | ES1 | ES0 | A16 | A15 | A14 | A13 |
         bit7  bit6  bit5  bit4  bit3  bit2  bit1  bit0
```

- `A13–A16`: 4 bits = 16 physical banks × 8 KiB = 128 KiB
- `ES0–ES3`: chip-select assertions (signal type, not address)

Hold this slide in your head. I will get this part wrong and have to come back to it.

The plan

What I asked the LLM to do

Roughly:

1. Read the R65C19 datasheet. Build a SLEIGH spec covering the whole ISA, including the indirect-mode trap.
2. Read the C40/L39 manual. Build the L3902-specific bits: peripheral symbols, vector table, memory map.
3. Write a Ghidra loader that handles $>$64 KiB banked images.
4. Write tests: SLEIGH compile, opcode-decode fixture, end-to-end firmware import.
5. Run it. Show me what the firmware does.

This is not "one prompt." Over four days I sent maybe 30 messages.

What the LLM had access to

- Three vendor PDFs (R65C19 data sheet, C40/L39 TRM, RC240VFC)
- The 128 KiB firmware blob
- A working Ghidra install at `/home/honx/ghidra`
- Shell, file editing, the SLEIGH compiler, headless analysis
- No internet. No prior code. No examples.

The session was Claude Code (Anthropic's CLI agent). Same pattern should work in any agentic setup with shell + file editing.

Building the SLEIGH spec

What SLEIGH is

Ghidra's processor description language. Defines:

- Endianness, address spaces
- Registers and flags
- Tokens (how opcodes are decomposed bit-by-bit)
- Constructors: "these bytes mean this disassembly + this p-code"

Built-in 6502 spec ships with Ghidra. *Not* reusable here: `(zp,X)` and `(zp),Y` are baked into the operand tables.

The R65C19 spec is one self-contained file, ∼600 lines.

Sample constructor: `STI #imm,$zp`

The R65C19's three-byte store-immediate-to-zero-page op:

```
# STI ($B2): 3 bytes - op, immediate value, ZP destination address.
:STI "#"imm8b, imm8 is op=0xB2; imm8b; imm8
{
    local addr:2 = imm8;
    *:1 addr = imm8b:1;
}
```

Read it left to right:
"`STI` #*value*, *address* matches when the opcode byte is `0xB2`, followed by an 8-bit value, then an 8-bit address. P-code: write the value to that address."

181 occurrences in the firmware. Rockwell did not skimp on this op.

Indirect-mode remap

The thing 65C02 gets silently wrong on R65C19:

```
# OP1 = operand class for cc=01 ALU instructions.
#   bbb=0  (zp)         R65C19 zero-page indirect (was (zp,X) on 6502)
#   bbb=4  (zp),X       R65C19 zp indirect, post-indexed by X (was (zp),Y)
OP1: (imm8)     is bbb=0; imm8        { addr:2 = imm8; tmp:2 = *:2 addr;
                                        export *:1 tmp; }
OP1: (imm8),X   is bbb=4 & X; imm8    { addr:2 = imm8; tmp:2 = *:2 addr;
                                        tmp = tmp + zext(X); export *:1 tmp; }
```

Two slots in the operand table that the LLM had to flip from the stock 6502 versions. Without this, every `LDA ($xx),X` in the firmware decodes as `LDA ($xx),Y` — silently wrong.

The opcode matrix

The R65C19 datasheet has a 16×16 opcode matrix. 256 cells, each labelled with mnemonic + addressing mode + cycle count.

The LLM's job: read the matrix from a 1992 photocopy quality PDF, expand each cell into a SLEIGH constructor.

What worked:

- All 256 opcodes decoded correctly — 0 unknown mnemonics in the firmware.
- Mnemonic lookup, encoding fields, byte counts: all OCR'd cleanly.

What didn't:

- Three opcodes the LLM couldn't read confidently: `JPI`, `ADD zp`, `ADD zp,X`. Documented as "unknown" in `open-points.md`. None appear in the reference firmware, so the gap doesn't bite.

The loader and the test suite

Loader: banked memory

Java loader, ∼150 lines. Recognises an L3902 firmware image by size constraint (multiple of 8 KiB up to 512 KiB) and:

- Maps the first 64 KiB at `$0000–$FFFF` as ROM
- Layers uninitialised blocks for the on-chip I/O, RAM pages, CRC
- Reads the reset vector at `$FFFE` and creates a `_reset` function there
- For images >64 KiB, drops each additional 64 KiB chunk in as a `BANK_HIGH_n` overlay block

Bank-aware cross-references — "after this `STI #imm,$1B`, `$6200` reads from `file:0x0200`" — are **not** implemented.
That's listed under open points.

Tests

Three of them, all green, all run in CI.

- `test_01_sleigh_compiles.sh` — recompile spec, fail on errors or unexpected warnings.
- `test_02_opcode_decode.sh` — generate a 64 KiB synthetic fixture with 43 hand-picked opcodes, import, assert that each address disassembles to the expected mnemonic.
- `test_03_e2e_firmware.sh` — import the real ELSA firmware, run auto-analysis, assert on 16 invariants.

```
=== test_01_sleigh_compiles.sh ===
  PASS SLEIGH spec compiles cleanly (2 warnings, 7461 byte .sla)
=== test_02_opcode_decode.sh ===
  PASS 43 opcode expectations satisfied
=== test_03_e2e_firmware.sh ===
  PASS 16 e2e assertions satisfied against the reference firmware
All 3 test(s) passed
```

The analysis

First decode

Reset vector: file `0xFFFE/F = c0 ff` → reset target `$FFC0`.

Disassembly at $FFC0:

```
$FFC0  78          SEI                ; mask all IRQs
$FFC1  B2 70 1B    STI #$70, $1B      ; BSR3 := $70 - bank-switch
$FFC4  4C 00 62    JMP $6200          ; jump into newly-mapped bank
```

This is the moment the spec "works."

- Three instructions disassembled cleanly.
- One of them is the R65C19 `STI` extension that no stock 6502 spec can decode.
- Auto-analysis finds 204 functions in 4 seconds.

What the firmware does, by string evidence

The image carries its own functional spec, in printable strings. The LLM dumped them, grouped them, and wrote them up.

`(C) ROCKWELL / ELSA Aachen` branding (1997-07-18 build)

`V42bis, TxSymRa Bd, TxPrecoding, V34PwrRed...` V.34 / V.42bis stats labels

`ROCKWELL;ADPCM;16` non-standard voice-mode codec

`OK, CONNECT, NO CARRIER, DIAL LOCKED...` AT result codes (Hayes/V.250)

`+FCON, +FHNG, +FDIS, +FCFR, +FNSC...` Class 2 fax (T.30)

`NO H324 DETECTED` H.324 video conferencing

`+VCON` V.253 voice connection

`Konfiguration:, RX-Gain:, Werte des fernen Modems` German diagnostic UI

Architecture, by code

Trampoline-driven dispatch.

204 functions, 2179 instructions in the boot-time view. Many follow a 6-byte template:

```
LDY $85
RND
STI #$03, $41          ; command opcode
STI #$12, $40          ; sub-command
JSR $E81B              ; central dispatcher
RTS
```

- One stub per AT command. ($41,$40) = command pair.
- Dispatcher at $E81B does the table lookup once, for everyone.
- STI alone accounts for 8.3% of the instruction stream.

The bug

The bug.

Where the LLM's confident wrongness met a human reviewer.

What the LLM said

After analysing the boot stub the LLM wrote, in the analysis document:

> *"The 128 KiB image is laid out so that **the first 64 KiB is what the L3902 sees while only ES0 is active**, and the second 64 KiB is what it sees once ES1, ES2 and ES3 are also asserted. . . . The boot stub bank-switches BSR3 to expose chip-2 in the $6000-$7FFF window, then jumps to $6200."*

This is the "two physical chips" theory. It is plausible, parses the BSR write, sounds technically literate.

It is also wrong.

What the human said

Eight hours after the analysis was committed, a friend with hands-on modem RE experience read it and emailed:

> *FFC1 B2 70 1B STI #70, $1B*
> *; Bank Select 3*
> *; 01110000*
> *; 6000-7FFF -> ROM 0000-1FFF*
>
> *d.h. bei 6200h finde ich dann den Code, der im Binärfile bei 0200 steht. Das erklärt vieles.*

"So at 6200h I find the code that's at 0200 in the binary file. That explains a lot."

What was actually at file 0x0200

Eight bytes:

```
file 0x0200:  B2 73 1F  4C B6 E7
              STI #$73, $1F          ; BSR7 := $73 - second-stage bank-switch!
              JMP $E7B6              ; into the newly-mapped bank
```

It's a second boot stub.

- Stage 1 ($FFC0): bank-switch BSR3, jump to $6200.
- Stage 2 ($6200, = file 0x0200): bank-switch BSR7, jump to $E7B6.
- Stage 3 ($E7B6, = file 0x67B6): the *real* hardware initialiser.

A three-stage boot trampoline, hidden in plain sight. The LLM never saw it because its memory model said 6200 wasn't where the bytes B2 73 1F lived.

The lesson

The LLM had read:

- The full BSR encoding.
- The 128 KiB file size.
- The L39 TRM's "up to 512 KiB external memory" claim.

It had *enough information* to derive the right model.
It instead constructed a confident, plausible, wrong one.

Specific failure mode: pattern-fit a coherent narrative around incomplete reasoning, present it as established fact.

Specific corrective: a human with hands-on experience reading the encoding from the bit pattern and saying "no, that's bank zero, the file offset is 0x0200."

The bit pattern was visible to both of us. Only one of us read it correctly.

What got fixed

After the correction, in one round of edits:

- Analysis writeup rewritten (∼70 lines changed) with the correct three-stage boot model.
- `docs/architecture.md` gained a "Banking is not in the SLEIGH spec" section explaining what SHOULD model banking (loader + analyser, not slaspec).
- `docs/open-points.md` BSR-aware-analysis bullet strengthened with the boot trampoline as motivation.
- E2E test grew 7 new assertions covering the second and third stages and the runtime IRQ vector table.

The repo now has *both* the original mistake (in git history) and the correction (with attribution in the commit message).

And then . . . a second email

Two days after the analysis was committed and corrected, a follow-up arrived with a screenshot. One table:

Bank	Cfg1	Cfg2	Cfg3	Cfg4
B0 ($0200)	ROM0	ROM8	ROM10	ROM18
B2 ($2000)	ROM2	ROMA	ROM12	ROM1A
B4 ($4000)	ROM4	ROMC	ROM14	ROM1C
B6 ($6000)	ROME	INX	ROM16	ROM1E
B8 ($8000)	RAM0	RAM0	RAM0	RAM0
BA ($A000)	RAM1	RAM1	RAM1	RAM1
BC ($C000)	RAM2	RAM2	RAM2	RAM2
BE ($E000)	ROM6	ROM6	ROM6	ROM6

"*Nach dem Reset wird die Konfiguration Cfg2 aktiviert.*"

What the second correction added

Before: "a single runtime memory map after init."

After: "four named configurations the firmware switches between."

- There are four functions called `switch_rom_cfg1()` through `switch_rom_cfg4()`. Each writes the eight BSR values for one configuration.
- Three of the BSR windows are *always* the same: 24 KiB of external SRAM at $8000-$DFFF, plus a persistent code bank ROM6 at $E000-$FFFF.
- The lower four windows ($0200-$7FFF) swap between four ROM groupings. The whole 128 KiB EPROM gets used: 8 KiB of code always present, plus 32+24+24+32 KiB swapped in by config.
- Reset puts the chip in Cfg2.

What this corrects: my "runtime memory map" was the post-reset Cfg2 state. I'd missed that there are three other states the firmware also uses.

Two corrections, one lesson

- **Round 1 (the BSR encoding):** I had "two physical chips". He had "read the bit pattern."
- **Round 2 (the four configurations):** I had "one runtime memory map". He had "four named configurations the firmware actually uses."

In both rounds, the corrective came from someone who had *worked on this kind of firmware before*.

The LLM had read a 119-page L39 manual and a 46-page R65C19 datasheet. What it didn't have was the embedded engineer's reflex of asking, "what does the firmware author's mental model look like?"

Honest assessment

What worked well

- **SLEIGH grammar and constructors** — mechanical, well-specified, large existing corpus. LLM strength.
- **Loader boilerplate** — 50 example loaders in Ghidra source. LLM had absorbed the API.
- **Test scaffolding** — shell drivers, headless scripts, fixture generation. Mostly boilerplate.
- **Documentation at multiple levels** — overview, usage, architecture, open-points. Tedious for humans, fast for an LLM.
- **Refactoring under spec** — "move this to a separate cspec file," "rename `I` to `Iflag` to avoid the threaded-code reg collision". Mechanical.

What didn't work well

- **Confident speculation about hardware semantics** — the BSR bug is the canonical example.
- **Inferring meaning from incomplete OCR** — three opcodes silently dropped because the matrix scan wasn't clean. Honest about the gap, but a human would have asked for a higher-resolution scan.
- **Cross-bank reasoning** — when the firmware's logical $6200 reads from a different file offset than the user thinks, the LLM's mental model drifted before it had a chance to be wrong on paper.
- **"Verifying" without actually verifying** — the LLM sometimes claimed it had checked something it had only plausibility-tested. Pre-commit, before the friend's email.

The collaboration model

Human

- Picks the target.
- Defines what “done” means.
- Verifies hardware semantics.
- Reads the bit patterns.
- Reviews the bigger claims.

LLM

- Reads the datasheet pages.
- Writes the boilerplate.
- Writes the tests.
- Writes the docs.
- Refactors on request.
- Surfaces inconsistencies.

What this is not: hand-the-LLM-a-PDF, get-a-Ghidra-module-back.

What this is: pair-programming with someone who works very fast, has read every reference doc, and has no engineering judgement. You provide the judgement.

The artefact

What's in the repo

github.com/honx/rockwell_l39_ghidra

- Full Ghidra processor module (drop-in)
- SLEIGH spec, ∼600 lines, all R65C19 ISA
- Java loader for banked images
- Three tests, all green, runnable
- Four docs (overview/usage/architecture/open-points)
- 350-line analysis of the ELSA firmware
- Reference firmware excluded by `.gitignore` (not ours to redistribute)

Apache 2.0. Patches welcome. The bug history is in the git log; if you want to see what the LLM got wrong before review, look at commit `20026e9^`.

Closing

- LLM-assisted RE works on real targets, with caveats.
- The caveats include a real failure mode that needs human review.
- "Real" here means: tested, documented, on a chip Ghidra had never heard of, four days from blank page to disassembled firmware.

If you remember one thing:

The LLM is a fast junior collaborator with no engineering judgement.
Treat it like one and it produces good work.
Treat it like a senior engineer and it ships confident bugs.

Questions?

honx@thinventions.de
github.com/honx/rockwell_l39_ghidra